Ad-Vocates Technical Specification

Project: Ad-Vocates

Team Members: Max Marsh, Dylan Shackleford

Document Version: 1.3

Date: October 30, 2026

Requirements Baseline: `Requirements_Document.md` **Version 1.0, October 30, 2026**

1. Version and Change Log

Version	Date	Change	Author
0.1	2026-10-30	Consolidated D0 architecture, D1 data model/API contract, D2 edge-case design, project constraints, and Week 8 implementation ownership into one technical specification draft.	Max Marsh, Dylan Shackleford
1.0	2026-10-30	Added component boundaries, data formats, stack decisions, constraint consequences, and full traceability to Requirements Document v1.0.	Max Marsh, Dylan Shackleford

This specification traces to **Requirements Document v1.0**. If a requirement changes, the requirements document version must be updated first and the traceability matrix in this document must then be revised.

2. Architecture Summary

Ad-Vocates is designed as a local browser extension with no required cloud backend. The D0 architecture divides the system into seven major components: **Dashboard**, **Settings**, **Filter Request**, **Local Storage**, **Statistic Recorder**, **Filter Updater**, and **Filter Validator**. The extension interacts with browser-provided extension APIs and stores required application data on the user's device.

The **Dashboard** is the user-facing component. It displays protection state, blocking statistics, estimated data savings, and filter-update status. User changes are sent to **Settings**, which validates changes and stores protection and allowlist data in **Local Storage**. When the browser attempts to load a resource, **Filter Request** receives the request information, checks whether protection is enabled, checks whether the page is allowlisted, and evaluates the request against the active filtering rules. The result is an `ALLOW` or `BLOCK` decision. A blocked request is reported to **Statistic Recorder**, which updates the blocked-request count and estimated bytes saved in **Local Storage**.

Filter maintenance follows a separate path. **Filter Updater** retrieves data from an approved external filter source over HTTPS. The downloaded data is passed to **Filter Validator**. Only validated rules are stored and made available to **Filter Request**. If retrieval or validation fails, the previous valid rule set remains active and the failure is recorded for status/error reporting. This design keeps normal browsing data local while still allowing filter definitions to be refreshed.

Architecture references:

- D0 System Diagram
- D0 High-Level Design
- D1 Data Model
- D1 Detailed Design and API Contract
- D2 Detailed Design

3. Component Interfaces and Data Formats

3.1 Component Boundary Table

Boundary	Inputs	Outputs	Format	Protocol / Mechanism	Error Behavior
Browser -> Filter Request	`requestId`, `requestUrl`, `pageHost`, `resourceType`	Request context for evaluation	JavaScript object / JSON-compatib	Chromium extension request/event interface	Invalid or unparseable URLs produce `INVALID_URL`;

			le payload	into extension code	the extension must not crash.
Dashboard -> Settings	`enabled`, optional allowlist changes	`success`, `updatedCount`	JSON-compatible message	Internal extension message or direct module call	Invalid settings/hostnames are rejected; storage failures return `STORAGE_ERROR`.
Settings -> Local Storage	`ApplicationSettings`, `AllowlistEntry` changes	Stored state or read result	Browser storage records	Browser local storage API	Failed writes do not silently change in-memory state; caller receives `STORAGE_ERROR`.
Local Storage -> Filter Request	Protection state, allowlist, active filter rules	Cached settings/rules used for evaluation	Objects / arrays / set-like lookup structures	Local browser storage read followed by in-memory cache	Missing filter data returns `FILTERS_UNAVAILABLE`; missing settings fall back to safe defaults.
Filter Request -> Statistic Recorder	Blocked-request event	Acknowledged statistic update	JSON-compatible internal event	Internal module call/message	A statistic-recording failure is logged but must not reverse an already made request decision.

Statistic Recorder -> Local Storage	Incremented `blockedTotal`, `estimatedBytes`	Stored `StatisticsSummary`	Structured local record	Browser local storage write	On failure, return/log `STORAGE_ERROR`; no duplicate count is added for the same request ID.
Dashboard -> Statistics data	No required input	`blockedTotal`, `estimatedBytes`	JSON-compatible response	`getStatistics` internal method	Storage read failure returns `STORAGE_ERROR` and Dashboard shows unavailable/error status.
Filter Updater -> Approved filter source	HTTPS filter request	Raw filter-list text and source metadata	Text over HTTPS	HTTPS GET through browser/network API	Network failure returns `NETWORK_ERROR`; current valid rules remain active.
Filter Updater -> Filter Validator	Raw filter-list data	Validated rules or rejection	Text input, structured `FilterRule[]` output	Internal module call	Invalid/malformed data returns `INVALID_FILTER_DATA`; rejected data is not activated.
Filter Validator -> Local Storage	Validated `FilterRule[]`, update metadata	Stored active rule set	Structured local data	Local storage / IndexedDB transaction as needed	Failed commit returns `STORAGE_ERROR`; previous valid rule set remains active.

Components -> ErrorLog	Error code/message and timestamp	Stored `ErrorLog` record	Structured local record	Local storage write	Logging failure must not cause a second user-facing failure loop.

3.2 Internal API / Endpoint Table

Ad-Vocates does not require a public web API. The D1 contract defines internal extension methods/messages that act as the component API.

Method	Required Input	Optional Input	Output	Errors	Traces To
`evaluateRequest`	`requestUrl: string, pageHost: string, resourceType: string, requestId: string`	None	`decision: "ALLOW" | "BLOCK", matchedRuleId: string | null`	`INVALID_URL, FILTERS_UNAVAILABLE, INTERNAL_ERROR`	REQ-01, REQ-02, REQ-04, NFR-01
`isAllowlisted`	`hostname: string`	None	`isAllowlisted: boolean`	`INVALID_HOSTNAME, STORAGE_ERROR`	REQ-02, REQ-03, CON-03
`updateSettings`	`enabled: boolean`	`allowlist: string[]`	`success: boolean, updatedCount: integer`	`INVALID_SETTING, INVALID_HOSTNAME, STORAGE_ERROR`	REQ-03, REQ-04,

					REQ-10
`getStatistics`	None	None	`blockedTotal: integer, estimatedBytes: integer`	`STORAGE_ERROR`	REQ-05, REQ-06
`refreshFilters`	None	None	`success: boolean, ruleCount: integer, updatedAt: string`	`NETWORK_ERROR, INVALID_FILTER_DATA, STORAGE_ERROR`	REQ-07, REQ-08, REQ-09

3.3 Core Message Payloads

Request Evaluation Input

```
{
  "method": "evaluateRequest",
  "requestUrl": "https://example-ad-domain.com/banner.js",
  "pageHost": "example.com",
  "resourceType": "script",
  "requestId": "req-1042"
}
```

Request Evaluation Output

```
{
  "decision": "BLOCK",
  "matchedRuleId": "rule-125"
}
```

Settings Update

```
{
  "method": "updateSettings",
  "enabled": true,
  "allowlist": ["example.com"]
}
```

Filter Refresh Result

```
{
  "success": true,
  "ruleCount": 10000,
  "updatedAt": "2026-10-30T16:00:00Z"
}
```

3.4 Local Data Schema

The D1 data model defines logical records. Primary/foreign-key labels describe relationships even if the selected browser store does not enforce relational foreign keys.

Record	Fields	Notes
`ApplicationSettings`	`settingsId: string` (PK), `enabled: boolean`	One current settings record is expected.
`AllowlistEntry`	`hostname: string` (PK), `settingsId: string` (FK)	Zero or more entries belong to `ApplicationSettings`; hostname is normalized before storage.
`FilterRule`	`ruleId: string` (PK), `ruleText: string`, `settingsId: string` (FK)	Zero or more active rules are associated with the settings/configuration context.

StatisticsSummary	statsId: string (PK), settingsId: string (FK), blockedTotal: integer, estimatedBytes: integer	One current summary is expected.
ErrorLog	errorId: string (PK), settingsId: string (FK), errorMsg: string, loggedAt: string	loggedAt uses ISO 8601 timestamp format.

3.5 Validation and Normalization Rules

- Hostnames are normalized to lowercase before allowlist comparison.
- Empty or invalid hostnames are not stored.
- Duplicate allowlist entries are not stored.
- A malformed request URL must not crash request processing.
- Duplicate or multiple matching filter rules must not cause one request to be counted more than once.
- If multiple rules match, the request is still blocked once.
- If protection is disabled, request filtering is skipped and the request is allowed.
- If the current hostname is allowlisted, normal blocking rules are skipped for that site.
- Invalid filter data is rejected before replacing the current active rules.
- A failed filter refresh preserves the last valid filter set.

4. Technology Stack

The stack below reflects the current repository design and the planned first-release deployment for Ad-Vocates.

Layer	Current Choice	Justification / Trace
Deployment	Local browser extension; Chromium-first baseline	Matches the D0/D1 deployment model, avoids a required backend, supports CON-01 local privacy, and is the current

		implementation assumption for REQ-01 through REQ-10.
Extension platform	Chromium Extension APIs; exact request-blocking API confirmed during T1 browser-support investigation	Browser-provided APIs are mature and avoid building browser-level request interception from scratch. Tied to REQ-01, NFR-01, and CON-01.
Core language	JavaScript for the initial implementation	Works directly with browser extension APIs, has no licensing cost, avoids an additional compile step, and is sufficient for the team's planned extension scope. Tied to REQ-01 through REQ-10 and NFR-01.
Dashboard UI	HTML, CSS, and JavaScript	Keeps the Dashboard lightweight and minimizes dependencies for the relatively small interface. Supports REQ-03, REQ-04, REQ-05, REQ-06, REQ-09, NFR-03, and CON-02.
Settings / small local records	Browser extension local storage API	Appropriate for enabled state, allowlist entries, summary statistics, and update/error metadata; keeps information local for CON-01 and supports REQ-03, REQ-04, and REQ-10.
Filter-rule storage	Browser storage for small rule sets; IndexedDB if the selected validated list exceeds practical extension-storage limits	D1 identified both browser storage and IndexedDB. The final split depends on OQ-02 and the selected list size. Supports REQ-07, REQ-08, REQ-10, and NFR-01.
In-memory lookup	Set/hash lookup for allowlist; grouped/indexed rule structures for filtering	D1 selects expected $O(1)$ allowlist checks and approximately $O(1 + k)$ candidate-rule lookup to support NFR-01.

External filter data	Existing maintained filter list from an approved HTTPS source; exact list pending OQ-01	Reuse avoids recreating a large filter corpus and supports REQ-07/REQ-08. License, update frequency, maturity, compatibility, and performance must be checked before distribution.
Source control / release artifacts	GitHub repository	Already used by the team for design, source, documentation, and planning. Supports maintainability and the course submission workflow.
Hosting	No required hosted backend for the first release	Reduces cost and privacy exposure and directly supports CON-01.

5. Known Constraints and Consequences

ID	Restated Constraint	What It Rules Out or Forces
CON-01	Ad-Vocates must protect privacy, minimize collected data, and verify external filter data before activation.	Forces local-first storage, HTTPS-only approved filter retrieval, validation before activation, and failure behavior that retains the previous valid rules. Rules out a design that uploads browsing history or request logs to an unnecessary remote backend.
CON-02	Protection choices and estimated data savings must be presented honestly and understandably.	Forces clear enabled/disabled and allowlist controls and an explicit “estimate” label for data-savings values. Rules out presenting estimated bytes as an exact measured savings value.
CON-03	Blocking can damage website usability, so users need a practical recovery path.	Forces an allowlist and protection controls and requires site-breakage cases to fail recoverably. Rules out a blocker that can only be disabled globally when a site breaks.

Additional implementation constraints from the current design:

- No cloud backend is required for the first release.
- The filter-list license must permit the team's intended use/distribution before the list is bundled or redistributed.
- The exact filter source and exact Chromium request-blocking API remain implementation decisions assigned to Dylan's T1/T3 work and must be resolved before the corresponding code is considered complete.
- Settings, allowlist data, statistics, and errors are stored locally and must remain consistent with the D1 schema.

6. Requirement-to-Design Traceability Matrix

The planned Week 10 test IDs are included below so each requirement, non-functional requirement, and constraint already has a defined verification target. No design component is blank.

Req ID	Description	Design Component (D0/D1/D2)	Status	Test Case
REQ-01	Block requests that match active rules when protection is enabled.	D0 Filter Request; D1 Request Filtering algorithm and `evaluateRequest`; D2 request edge cases	Designed	TC-001
REQ-02	Bypass blocking for allowlisted sites.	D0 Settings + Filter Request; D1 Allowlist Checking and `isAllowlisted`; D2 site-breakage recovery	Designed	TC-002
REQ-03	Add/remove/validate/persist allowlist entries.	D0 Settings + Local Storage; D1 `AllowlistEntry` + `updateSettings`; D2 invalid/duplicate hostname cases	Designed	TC-003

REQ-04	Enable/disable global protection.	D0 Settings + Filter Request; D1 `ApplicationSettings.enabled` + `updateSettings`; D2 disabled-protection case	Designed	TC-004
REQ-05	Count each blocked request once and update displayed total.	D0 Statistic Recorder + Dashboard + Local Storage; D1 StatisticsSummary; D2 duplicate/multiple-match handling	Designed	TC-005
REQ-06	Display estimated data savings and label it as an estimate.	D0 Statistic Recorder + Dashboard; D1 `estimatedBytes`; D2 statistics/display handling	Designed	TC-006
REQ-07	Refresh filtering data without reinstalling.	D0 Filter Updater + Filter Validator; D1 `refreshFilters`; D2 update/retry handling	Designed	TC-007
REQ-08	Validate new filter data and preserve previous rules on failure.	D0 Filter Validator + Filter Updater + Local Storage; D1 refresh errors; D2 failure edge cases	Designed	TC-008
REQ-09	Display filter-update status and readable failure state.	D0 Dashboard + Filter Updater; D1 refresh result/error contract; D2 update-status handling	Designed	TC-009
REQ-10	Persist settings, filter metadata, and summary stats across restart.	D0 Local Storage; D1 data model; D2 persistence/error handling	Designed	TC-010
NFR-01	Meet request-filtering latency target with ~10,000 rules.	D0 Filter Request; D1 indexed/grouped rule-matching design; D2 performance consideration	Planned	TC-011

NFR-02	Use secure approved filter sources and do not transmit browsing records to unrelated services.	D0 Filter Updater + Local Storage; D1 build/reuse and local-storage decisions; D2 security handling	Planned	TC-012
NFR-03	Meet timed usability target for protection, allowlist, and statistics tasks.	D0 Dashboard + Settings; D1 UI/technology choices; D2 recovery/interaction cases	Planned	TC-013
NFR-04	Meet stress, failed-update, and persistence reliability thresholds.	D0 Filter Request + Filter Updater + Local Storage; D1 errors/edge cases; D2 failure handling	Planned	TC-014
CON-01	Security/privacy and verified update constraint.	D0 Filter Updater + Filter Validator + Local Storage; D1 local-first design; D2 update failure paths	Designed	TC-015
CON-02	Ethical transparency for settings and estimated data.	D0 Dashboard + Settings + Statistic Recorder; D1 statistics fields; D2 UI/status behavior	Designed	TC-016
CON-03	Website-usability recovery constraint.	D0 Settings + Filter Request; D1 allowlist algorithm; D2 breakage recovery cases	Designed	TC-017

7. Advisor Review Record

Review Date	Documents / Topics Discussed	Changes Produced
September 18, 2026	Ad-Vocates project scope, team contract, team responsibilities, and overall implementation direction.	Project and team contract approved. The project remained focused on the original privacy-focused browser extension scope.

8. Repository References

- Requirements Document
- Project Description
- User Stories
- D0 High-Level Design
- D1 Detailed Design
- D2 Detailed Design
- Task List
- Week 8 Planning Package